```
#include <SoftwareSerial.h>
#include <TinyGPS.h>

// Define the RX and TX pins for the SoftwareSerial interface
#define GPS_TX_PIN 2
#define GPS_RX_PIN 3

// Create a SoftwareSerial object for communication with the GPS module
SoftwareSerial gpsSerial(GPS_RX_PIN, GPS_TX_PIN);

// Create a TinyGPS object
TinyGPS gps;

void setup() {
  // Initialize serial communication with the computer
  Serial.begin(9600);

  // Initialize serial communication with the GPS module
  gpsSerial.begin(9600);
}

void loop() {
  // Read data from the GPS module
  while (gpsSerial.available() > 0) {
    if (gps.encode(gpsSerial.read())) {
      // If a valid NMEA sentence is decoded, extract and print the data
      float latitude, longitude;
      unsigned long fix_age;
      gps.f_get_position(&latitude, &longitude, &fix_age);

      Serial.print("Latitude: ");
      Serial.println(latitude, 6);
      Serial.print("Longitude: ");
      Serial.println(longitude, 6);
    }
  }
}
```

Fig. 6: Final code snippet

data is dumped to the serial port to be seen on the serial monitor screen. The uploaded source code can be seen on the serial monitor. Fig. 5 shows the raw data code snippet.

The data stream can be observed by opening he serial port. One of the lines might look like: $GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47 In this example, $GPGGA is the prefix indicating the type of data (in this case, global positioning system fixes the data). The subsequent fields contain information such as time (123519), latitude (4807.038 N), longitude (01131.000 E), number of satellites in view (08), altitude (545.4 metres), etc.

To decode NMEA sentences in Arduino, libraries like TinyGPS can be used. We just saw a basic example of how TinyGPS might be used to decode a NMEA sentence. Fig. 6 shows the final code's snippet.

In this code, the TinyGPS library is used to decode NMEA sentences received from the GPS module via SoftwareSerial. Inside the loop() function, characters are continuously read from the GPS module's serial interface and passed to the gps.encode() function. When a complete NMEA sentence is received and successfully decoded, the latitude and longitude are extracted using gps.f_get_position() and printed to the serial monitor.

EFY Note
The source code of this project is available for free download at *www.electronicsforu.com*

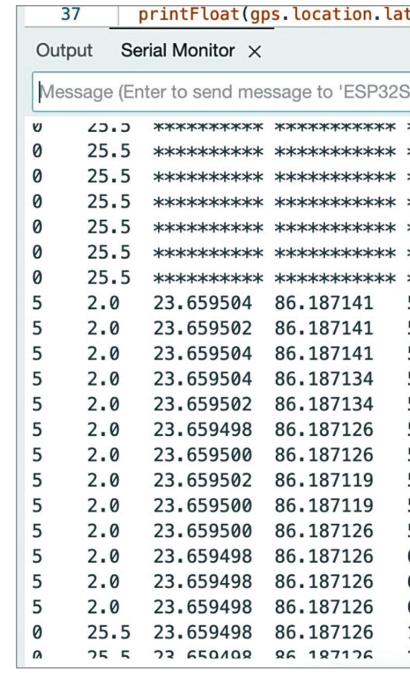

Fig. 7: Serial monitor showing GPS data

Testing

After uploading the source code into the Arduino board and interconnecting the Arduino Uno and NEO-6M module, the circuit is ready to use. Now power the module and open the serial monitor. The location can be mapped in latitude and longitude. Copy the latitude and longitude to Google Maps to view the location on maps. Fig. 7 shows the serial monitor showing GPS data. EFY


Rajesh Rathakrishnan is an embedded specialist with a passion for technology in IoT. He holds a degree in Bachelor of Engineering and has written extensively on topics ranging from embedded wireless IoT to cloud technologies.